



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina Análise
Numérica**

**Relatório de implementações realizadas
por Ronaldo Ribeiro Porto Filho**

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

**Ilhéus – BA
2026**

ÍNDICE

ÍNDICE.....	2
Linguagem(ns) Escolhida(s) e justificativas.....	5
Método de Euler.....	7
Estratégia de Implementação:.....	7
Estrutura dos Arquivos de Entrada/Saída.....	7
Dificuldades enfrentadas.....	8
Método de Euler Modificado.....	9
Estratégia de Implementação:.....	9
Estrutura dos Arquivos de Entrada/Saída.....	9
Dificuldades enfrentadas.....	10
Método de Heun.....	11
Estratégia de Implementação:.....	11
Estrutura dos Arquivos de Entrada/Saída.....	11
Dificuldades enfrentadas.....	12
Método de Ralston.....	13
Estratégia de Implementação:.....	13
Estrutura dos Arquivos de Entrada/Saída.....	13
Dificuldades enfrentadas.....	14
Método de Runge-Kutta de 4º ordem.....	15
Estratégia de Implementação:.....	15
Estrutura dos Arquivos de Entrada/Saída.....	15
Dificuldades enfrentadas.....	16
Problema Teste 1 - Página 423 Questão 12.3.....	17
Entrada 1 – para Runge-Kutta de 4º ordem.....	17
Saída – Runge-Kutta de 4º ordem (Entrada 1).....	18
Problema Teste 2 - Página 425 Questão 12.10.....	23
Entrada 1 – para Euler.....	23
Saída – Euler (Entrada 1).....	24
Problema Teste 3 - Página 426 Questão 12.16.....	26
Entrada 1 – para Runge-Kutta de 4º ordem.....	26
Entrada 2 – para Runge-Kutta de 4º ordem.....	26
Saída – Runge-Kutta de 4º ordem (Entrada 1).....	27
Saída – Runge-Kutta de 4º ordem (Entrada 2).....	29
Método do Shooting (Tiro).....	31
Estratégia de Implementação:.....	31
Estrutura dos Arquivos de Entrada/Saída.....	31
Dificuldades enfrentadas.....	32
Método das Diferenças Finitas.....	33
Estratégia de Implementação:.....	33

Estrutura dos Arquivos de Entrada/Saída.....	33
Dificuldades enfrentadas.....	34
Problema Teste 4 - Questão fornecida no slide do professor.....	35
Entrada 1 – para Shooting.....	35
Saída – Shooting (Entrada 1).....	36
Problema Teste 5 - Questão fornecida no slide do professor.....	37
Entrada 1 – para Diferenças Finitas.....	37
Saída – Diferenças Finitas (Entrada 1).....	38
Problema Teste 6 - Modelo tipo SEIAHR simples para simulação de quadro epidêmico hipotético.....	39
Considerações Finais.....	40

Linguagem(ns) Escolhida(s) e justificativas

A linguagem escolhida para o desenvolvimento de todas as implementações numéricas de Problemas de Valor Inicial (PVI) e Problemas de Valor de Contorno (PVC) deste relatório foi o Python. A decisão principal baseou-se em sua sintaxe expressiva e de alto nível, que oferece uma transcrição quase direta da notação matemática formal para a lógica computacional, reduzindo significativamente a margem de erros estruturais durante a tradução das Equações Diferenciais Ordinárias (EDOs).

Uma diretriz metodológica central neste trabalho foi a abstenção total de bibliotecas matemáticas de terceiros, como NumPy ou SciPy. O objetivo dessa restrição foi garantir que a essência iterativa e algébrica de cada algoritmo fosse implementada integralmente do zero. Desde os cálculos de médias ponderadas de inclinação no método de Runge-Kutta de 4ª Ordem, até a resolução direta de matrizes para sistemas lineares tridiagonais no método das Diferenças Finitas (através do Algoritmo de Thomas), toda a arquitetura foi desenvolvida em Python puro. Para as operações trigonométricas e exponenciais básicas que compõem as EDOs avaliadas, recorreu-se exclusivamente ao módulo nativo `math`.

Outro fator decisivo para a escolha da linguagem foi a robustez e simplicidade no tratamento de strings e manipulação de arquivos de texto nativos (`entrada.txt` e `saida.txt`). Foi construído um motor de avaliação matemática dinâmica valendo-se da função `eval()`, que operou estritamente dentro de um escopo léxico seguro. Ao injetar um dicionário de ambiente restrito que mapeava os caracteres literais das variáveis dependentes e independentes para seus respectivos valores flutuantes na iteração atual, o programa tornou-se capaz de interpretar e processar funções inseridas pelo usuário em tempo de execução. Isso eliminou a necessidade de alterar o código-fonte a cada novo problema-teste de EDO submetido.

Por fim, os recursos de formatação de cadeias de caracteres do Python mostraram-se fundamentais para a geração dos relatórios de processamento. Como os métodos desenvolvidos exigem acompanhamento minucioso — seja rastreando os coeficientes intermediários em algoritmos preditores-corretores ou documentando a calibração de malha e ajuste de ângulo no Método do Tiro (Shooting) — a

linguagem permitiu estruturar os arquivos de saída como verdadeiros memoriais de cálculo passo a passo, facilitando a auditoria teórica dos resultados numéricos.

Método de Euler

Estratégia de Implementação:

A implementação do Método de Euler foi estruturada de forma direta e iterativa, refletindo sua natureza como o procedimento de passo único mais fundamental para a resolução de Problemas de Valor Inicial (PVI). A base algorítmica fundamenta-se na premissa teórica do método: assumir que a inclinação da reta tangente no ponto atual se mantém constante ao longo de todo o intervalo do passo h .

Computacionalmente, o processo é orquestrado por um laço de repetição rigorosamente delimitado pelo número total de iterações n . A cada ciclo, a rotina de avaliação matemática processa a expressão inserida para calcular a taxa de variação instantânea $f(x_i, y_i)$. De posse desse coeficiente angular, a projeção do valor subsequente da variável dependente ocorre através da fórmula explícita $y_{i+1} = y_i + h \cdot f(x_i, y_i)$, enquanto a variável independente avança linearmente ($x_{i+1} = x_i + h$).

Para assegurar a rastreabilidade da solução contínua, as coordenadas originais e todas as subseqüentes são armazenadas de forma sequencial em uma estrutura de lista na memória. Essa organização em tuplas permite não apenas retroalimentar as variáveis do laço para a iteração seguinte, mas também a exportação integral da trajetória da curva aproximada ao término da execução do algoritmo.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): O arquivo de leitura requer estritamente cinco linhas, contendo exclusivamente os valores operacionais e dispensando quaisquer rótulos ou descrições textuais. A ordem obrigatória de parâmetros estabelecida inicia com a expressão da função diferencial $f(x, y)$ (em notação de string executável), seguida pelo valor numérico da condição inicial para a variável independente (x_0), o valor numérico da condição inicial para a variável dependente (y_0), o tamanho do passo de incremento (h) e, por fim, o número total de iterações a serem realizadas (n).

Arquivo de Saída (saida.txt): O documento gerado foi projetado para atuar como um memorial de cálculo transparente. A primeira seção consiste em um

cabeçalho de confirmação, ecoando a EDO e as condições de contorno fornecidas. Na seção principal, o algoritmo imprime a evolução passo a passo, exibindo o valor exato da inclinação $f(x, y)$ processada e a projeção do novo y a cada iteração.

Na consolidação dos resultados, a apresentação da malha de coordenadas segue um padrão visual limpo. A lista de pontos processados é formatada e exibida em uma única estrutura contínua, utilizando exclusivamente parênteses para denotar os pares ordenados, no formato (x, y) , garantindo uma leitura matemática padronizada da curva encontrada.

Dificuldades enfrentadas

A principal dificuldade na implementação do Método de Euler concentrou-se na lógica necessária para tornar o algoritmo dinâmico. Como a equação diferencial é extraída do arquivo entrada.txt como uma string de texto puro — em uma estrutura estrita e sem rótulos que exige tipagem rigorosa dos parâmetros —, o desafio foi convertê-la em uma instrução executável a cada iteração sem o uso de bibliotecas simbólicas de terceiros. Para contornar isso, aplicou-se a função nativa `eval()` atrelada a um dicionário de ambiente seguro, responsável por injetar de forma isolada os valores atualizados de x e y a cada novo cálculo. Em conjunto com esse desafio de estruturação do código, a própria natureza de primeira ordem do método evidenciou sua inerente suscetibilidade a erros de truncamento numérico, o que exigiu um monitoramento cauteloso do tamanho do passo h e da aritmética de ponto flutuante para garantir a validade dos resultados gravados no memorial de cálculo.

Método de Euler Modificado

Estratégia de Implementação:

A implementação do Método de Euler Modificado (frequentemente referido na literatura como Método do Ponto Médio) foi concebida como uma evolução natural do método de Euler simples, objetivando mitigar o erro de truncamento por meio de uma avaliação mais refinada da derivada.

A estratégia algorítmica desenvolvida adota um esquema de "meio passo". A cada iteração do laço principal, o código primeiramente utiliza o avaliador dinâmico para calcular a inclinação no ponto inicial do intervalo, armazenando-a na variável k_1 . Em vez de usar essa taxa para avançar o passo inteiro, o algoritmo projeta analiticamente as coordenadas exatas do centro do intervalo, gerando os valores transitórios $x_{\text{MÉDIO}}$ (avançando $h/2$) e $y_{\text{MÉDIO}}$ (projetado via k_1).

É neste ponto central que a rotina avalia novamente a EDO para descobrir a inclinação corrigida, denominada k_2 . O avanço definitivo da variável dependente consolida-se baseando-se unicamente nessa inclinação central, através da equação $y_{i+1} = y_i + h \cdot k_2$. Todo esse fluxo foi codificado de maneira estritamente sequencial, reaproveitando a mesma arquitetura de interpretação de strings da biblioteca math construída no método anterior, o que preservou a independência de pacotes externos e a eficiência do processamento.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): O arquivo de leitura requer estritamente cinco linhas, contendo exclusivamente os valores operacionais e dispensando quaisquer rótulos ou descrições textuais. A ordem obrigatória de parâmetros estabelecida inicia com a expressão da função diferencial $f(x, y)$ (em notação de string executável), seguida pelo valor numérico da condição inicial para a variável independente (x_0), o valor numérico da condição inicial para a variável dependente (y_0), o tamanho do passo de incremento (h) e, por fim, o número total de iterações a serem realizadas (n).

Arquivo de Saída (saida.txt): Após o cabeçalho de identificação do PVI, o arquivo documenta iterativamente os cálculos intermediários de cada etapa. Ele registra o valor da inclinação inicial (k_1), expõe as coordenadas exatas estimadas

para o ponto médio ($x_{\text{MÉDIO}}$ e $y_{\text{MÉDIO}}$) e revela a inclinação central (k_2), demonstrando a substituição numérica que originou o novo valor de y . Ao término das iterações, a totalidade das coordenadas geradas é consolidada em uma única estrutura contínua. A lista é formatada visualmente por meio de pares ordenados do tipo (x, y) , entregando a curva resultante pronta para análise analítica ou plotagem gráfica.

Dificuldades enfrentadas

A principal dificuldade técnica na implementação do Método de Euler Modificado concentrou-se na distinção conceitual e algorítmica entre esta abordagem e o Método de Heun. Como ambos representam melhorias de segunda ordem em relação ao método simples e são frequentemente confundidos na literatura, foi imperativo garantir que o código refletisse estritamente a mecânica do Ponto Médio — avaliando a inclinação central em $x + h/2$ — em vez de realizar uma média das inclinações nas extremidades do passo. Traduzir essa precisão teórica para o algoritmo exigiu um gerenciamento cauteloso de variáveis temporárias ($x_{\text{MÉDIO}}$ e $y_{\text{MÉDIO}}$), de modo a projetar as coordenadas fracionárias e injetá-las no interpretador dinâmico sem subscrever acidentalmente os valores originais do laço de repetição. Uma vez assegurado esse rigor no fluxo das operações, o método demonstrou-se altamente estável e pôde ser integrado de forma fluida, sem demandar alterações na infraestrutura padronizada de entrada e saída.

Método de Heun

Estratégia de Implementação:

A implementação do Método de Heun foi estruturada sob a clássica abordagem de predição e correção, configurando-se como uma evolução direta para refinar a estabilidade do método de Euler. O algoritmo computacional foi desenhado para executar esse processo em duas etapas bem definidas a cada ciclo do laço de repetição.

No estágio preditor, a rotina calcula a inclinação no ponto inicial do intervalo, armazenando-a como f_{xy1} (ou k_1), e utiliza a lógica de Euler para estimar provisoriamente onde estará o ponto final do passo, gerando um y_{predito} . Em seguida, começa o estágio corretor, o interpretador dinâmico avalia a equação diferencial exatamente nessas coordenadas futuras estimadas, descobrindo a segunda inclinação f_{xy2} (ou k_2).

O avanço definitivo da variável dependente é então consolidado aplicando-se a média aritmética entre a inclinação inicial e a predita, multiplicada pelo passo h . Ao reaproveitar a arquitetura modular da função de avaliação, o método garante a mesma flexibilidade de interpretação de strings matemáticas dos algoritmos anteriores, mantendo a coesão do projeto e a independência de bibliotecas externas complexas.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): O arquivo de leitura requer estritamente cinco linhas, contendo exclusivamente os valores operacionais e dispensando quaisquer rótulos ou descrições textuais. A ordem obrigatória de parâmetros estabelecida inicia com a expressão da função diferencial $f(x, y)$ (em notação de string executável), seguida pelo valor numérico da condição inicial para a variável independente (x_0), o valor numérico da condição inicial para a variável dependente (y_0), o tamanho do passo de incremento (h) e, por fim, o número total de iterações a serem realizadas (n).

Arquivo de Saída (saida.txt): O memorial de cálculo gerado adapta-se para documentar a mecânica de dois estágios do método. Após a impressão do cabeçalho com as condições do problema, o algoritmo detalha cada iteração

registrando explicitamente a inclinação atual (k_1) e a inclinação predita (k_2), além da coordenada x onde a predição ocorreu, demonstrando o valor resultante y_{prox} . Ao final da execução, todos os pontos validados são formatados e organizados de forma contínua em pares ordenados do tipo (x, y) , entregando a malha da curva completa e pronta para visualização.

Dificuldades enfrentadas

A principal dificuldade na implementação do Método de Heun concentrou-se na orquestração rigorosa de sua arquitetura de dois estágios (preditor e corretor). Diferentemente do método de Euler simples, foi necessário garantir que o cálculo da segunda inclinação (k_2) fosse alimentado no avaliador dinâmico estritamente com as coordenadas futuras estimadas (x_{prox} e y_{predito}), exigindo um sequenciamento meticuloso das variáveis dentro do laço de repetição. Qualquer desatenção na ordem das operações — como a atualização prematura da variável independente ou a sobrescrita acidental da inclinação inicial (k_1) antes do cálculo da média aritmética — invalidaria a lógica do algoritmo. Adicionalmente, exigiu-se cautela teórica para não fundir a mecânica desta média das inclinações (avaliadas nas extremidades do passo) com a lógica central do Ponto Médio (Euler Modificado), assegurando que o código refletisse a exata identidade matemática do algoritmo de Heun.

Método de Ralston

Estratégia de Implementação:

O Método de Ralston foi implementado como uma variação otimizada da família de métodos de Runge-Kutta de segunda ordem, desenhado especificamente para minimizar o limite do erro de truncamento local. A estratégia algorítmica afasta-se das avaliações no centro ou no final do intervalo, buscando a segunda estimativa de inclinação exatamente a 3/4 do caminho do passo de integração.

No código, o fluxo computacional foi estruturado para refletir essa precisão fracionária. Primeiro, o avaliador dinâmico captura a inclinação no ponto de partida, armazenando o coeficiente k_1 . Em seguida, o algoritmo projeta analiticamente as coordenadas em 75% do passo, gerando $x_{3/4} = x_{ATUAL} + 0,75h$ e estimando $y_{3/4} = y_{ATUAL} + 0,75h \cdot k_1$. É nesse ponto fracionário específico que a EDO é reavaliada para a obtenção da inclinação k_2 .

O avanço da variável dependente é consolidado por uma média ponderada estrita, onde a inclinação inicial (k_1) recebe peso de 1/3, e a inclinação projetada (k_2) recebe peso de 2/3. O novo valor é definido pela fórmula $y_{PROX} = y_{ATUAL} + (h/3) \cdot (k_1 + 2k_2)$.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): O arquivo de leitura requer estritamente cinco linhas, contendo exclusivamente os valores operacionais e dispensando quaisquer rótulos ou descrições textuais. A ordem obrigatória de parâmetros estabelecida inicia com a expressão da função diferencial $f(x, y)$ (em notação de string executável), seguida pelo valor numérico da condição inicial para a variável independente (x_0), o valor numérico da condição inicial para a variável dependente (y_0), o tamanho do passo de incremento (h) e, por fim, o número total de iterações a serem realizadas (n).

Arquivo de Saída (saida.txt): O relatório de execução foi personalizado para expor a mecânica da ponderação fracionária do método de Ralston. Após exibir as condições de contorno, o documento detalha o laço de repetição, registrando explicitamente a inclinação inicial (k_1) e enfatizando a obtenção de k_2 "em 3/4 do caminho". O cálculo de y_{PROX} é impresso de modo transparente, demonstrando a

substituição dos pesos na fórmula de atualização. Finalizando a execução, as coordenadas definitivas são aglutinadas e exibidas sequencialmente no formato (x, y), compondo a malha da curva resultante de forma limpa e padronizada.

Dificuldades enfrentadas

A principal dificuldade técnica na implementação do Método de Ralston residiu na transposição exata de seus coeficientes fracionários específicos ($3/4$, $1/3$ e $2/3$) para a lógica computacional, exigindo cautela para minimizar a perda de precisão inerente à ordem das operações na aritmética de ponto flutuante. Foi necessário um rigoroso controle para garantir que o coeficiente k_2 fosse obtido avaliando-se a função EDO estritamente nas coordenadas projetadas em 75% do passo de integração, o que demandou atenção redobrada ao aplicar o fator de $0,75h$ dentro dos parâmetros do avaliador dinâmico. Além disso, a estruturação da fórmula de atualização exigiu precisão algébrica na distribuição dos pesos — assegurando que a inclinação projetada (k_2) recebesse o dobro do peso da inclinação inicial (k_1) na soma final. Superar esse detalhamento rigoroso foi fundamental para distinguir claramente o algoritmo de outras abordagens de segunda ordem, como o método de Heun, garantindo que o código preservasse as propriedades teóricas de minimização do erro de truncamento local que caracterizam a formulação de Ralston.

Método de Runge-Kutta de 4º ordem

Estratégia de Implementação:

A estratégia de implementação do Método de Runge-Kutta de 4ª Ordem (RK4) fundamenta-se na busca por alta precisão e robustez na resolução numérica de EDOs, configurando-se como o padrão-ouro desta classe de algoritmos computacionais. O código foi estruturado para realizar quatro avaliações distintas da função derivada por passo de integração, capturando de maneira altamente refinada o comportamento da curva.

Sequencialmente, a rotina calcula a inclinação exata no ponto inicial do intervalo, armazenando-a como k_1 . A partir dessa base, o algoritmo estima duas inclinações no ponto médio do passo ($x_{ATUAL} + h/2$): a primeira (k_2) utiliza a projeção de k_1 , e a segunda (k_3) refina a estimativa central utilizando a projeção de k_2 . Por fim, a inclinação no extremo final do passo (k_4) é obtida utilizando a projeção de k_3 .

O avanço definitivo da variável dependente ocorre por meio da fórmula clássica de média ponderada, na qual as inclinações centrais (k_2 e k_3) recebem peso duplo, dividindo-se o somatório dos coeficientes por seis ($y_{PROX} = y_{ATUAL} + (h/6) \cdot (k_1 + 2k_2 + 2k_3 + k_4)$). Toda essa mecânica foi encapsulada na função modular do projeto (`metodo_runge_kutta_4`), reaproveitando o avaliador dinâmico de strings matemáticas e garantindo extrema precisão sem a necessidade de importar bibliotecas algébricas externas.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (`entrada.txt`): O arquivo de leitura requer estritamente cinco linhas, contendo exclusivamente os valores operacionais e dispensando quaisquer rótulos ou descrições textuais. A ordem obrigatória de parâmetros estabelecida inicia com a expressão da função diferencial $f(x, y)$ (em notação de string executável), seguida pelo valor numérico da condição inicial para a variável independente (x_0), o valor numérico da condição inicial para a variável dependente (y_0), o tamanho do passo de incremento (h) e, por fim, o número total de iterações a serem realizadas (n).

Arquivo de Saída (`saida.txt`): O relatório de execução foi adaptado para atuar como um memorial de cálculo exaustivo, refletindo o alto rigor algébrico do RK4. Em

cada iteração detalhada no documento, o sistema expõe explicitamente os valores calculados para as quatro estimativas de inclinação (k_1 , k_2 , k_3 e k_4). Logo abaixo, o registro descreve a substituição algébrica desses coeficientes na fórmula da média ponderada, evidenciando como o valor de y_{PROX} foi obtido. Ao encerrar o laço de repetição, a totalidade das coordenadas consolidadas é agrupada em uma malha contínua formatada em pares ordenados do tipo (x, y) , pronta para análises ou plotagem gráfica.

Dificuldades enfrentadas

A principal dificuldade na implementação do Método de Runge-Kutta de 4ª Ordem concentrou-se na gestão rigorosa de suas múltiplas variáveis intermediárias e no aninhamento de suas avaliações. Por ser um algoritmo que exige o cálculo sequencial de quatro inclinações distintas (k_1 , k_2 , k_3 e k_4), qualquer equívoco mínimo na passagem de parâmetros para o avaliador dinâmico — como a aplicação incorreta do incremento fracionário $h/2$ nas coordenadas centrais para obter k_2 e k_3 , ou a utilização da inclinação errada para projetar a variável dependente — propagaria erros silenciosos que comprometeriam a alta precisão característica do método. Adicionalmente, estruturar a equação final de atualização exigiu extrema atenção para garantir que a média ponderada refletisse perfeitamente os pesos teóricos clássicos divididos por seis. Uma vez superado esse desafio algorítmico de coordenar as projeções sucessivas dentro do laço de repetição, o método confirmou sua robustez, atuando inclusive como um referencial de comparação seguro para validar o nível de erro de truncamento dos métodos de ordem inferior desenvolvidos no projeto.

Problema Teste 1 - Página 423 Questão 12.3

12.3 - Suponha que o corpo descrito no problema [12.2](#) está sujeito a uma resistência do ar igual a duas vezes a velocidade. A equação diferencial agora é:

$$\frac{dv}{dt} = \frac{2000 - 2v}{200 - t}.$$

Se o corpo está em repouso em $t = 0$, a solução exata é $v = 10t - 40t^2$, de modo que $v(50) = 437.50$. Resolva o problema numericamente usando um método de Runge-Kutta de ordem 4.

Entrada 1 - para Runge-Kutta de 4º ordem

```
1 (2000 - 2*y) / (200 - x)
2 0
3 0
4 1
5 50
```


Saída – Runge-Kutta de 4º ordem (Entrada 1)

```
1  ===== RELATORIO: METODO DE RUNGE-KUTTA 4A ORDEM =====
2
3  Equacao Diferencial: dy/dx = (2000 - 2*y) / (200 - x)
4  Condicoes Iniciais: x0 = 0.0, y0 = 0.0
5  Passo (h) = 1.0 | Iteracoes (n) = 50
6
7  --- INICIANDO ITERACOES ---
8  Iteracao 0: x = 0.0000, y = 0.0000
9  Iteracao 1:
10     k1 = 10.0000 | k2 = 9.9749 | k3 = 9.9751 | k4 = 9.9500
11     y_prox = 0.0000 + (1.0/6) * (10.0000 + 2*9.9749 + 2*9.9751 + 9.9500) = 9.9750
12  Iteracao 2:
13     k1 = 9.9500 | k2 = 9.9249 | k3 = 9.9251 | k4 = 9.9000
14     y_prox = 9.9750 + (1.0/6) * (9.9500 + 2*9.9249 + 2*9.9251 + 9.9000) = 19.9000
15  Iteracao 3:
16     k1 = 9.9000 | k2 = 9.8749 | k3 = 9.8751 | k4 = 9.8500
17     y_prox = 19.9000 + (1.0/6) * (9.9000 + 2*9.8749 + 2*9.8751 + 9.8500) = 29.7750
18  Iteracao 4:
19     k1 = 9.8500 | k2 = 9.8249 | k3 = 9.8251 | k4 = 9.8000
20     y_prox = 29.7750 + (1.0/6) * (9.8500 + 2*9.8249 + 2*9.8251 + 9.8000) = 39.6000
21  Iteracao 5:
22     k1 = 9.8000 | k2 = 9.7749 | k3 = 9.7751 | k4 = 9.7500
23     y_prox = 39.6000 + (1.0/6) * (9.8000 + 2*9.7749 + 2*9.7751 + 9.7500) = 49.3750
24  Iteracao 6:
25     k1 = 9.7500 | k2 = 9.7249 | k3 = 9.7251 | k4 = 9.7000
26     y_prox = 49.3750 + (1.0/6) * (9.7500 + 2*9.7249 + 2*9.7251 + 9.7000) = 59.1000
27  Iteracao 7:
28     k1 = 9.7000 | k2 = 9.6749 | k3 = 9.6751 | k4 = 9.6500
29     y_prox = 59.1000 + (1.0/6) * (9.7000 + 2*9.6749 + 2*9.6751 + 9.6500) = 68.7750
30  Iteracao 8:
31     k1 = 9.6500 | k2 = 9.6249 | k3 = 9.6251 | k4 = 9.6000
32     y_prox = 68.7750 + (1.0/6) * (9.6500 + 2*9.6249 + 2*9.6251 + 9.6000) = 78.4000
33  Iteracao 9:
34     k1 = 9.6000 | k2 = 9.5749 | k3 = 9.5751 | k4 = 9.5500
35     y_prox = 78.4000 + (1.0/6) * (9.6000 + 2*9.5749 + 2*9.5751 + 9.5500) = 87.9750
36  Iteracao 10:
37     k1 = 9.5500 | k2 = 9.5249 | k3 = 9.5251 | k4 = 9.5000
```

```

38 | y_prox = 87.9750 + (1.0/6) * (9.5500 + 2*9.5249 + 2*9.5251 + 9.5000) = 97.5000
39 | Iteracao 11:
40 | k1 = 9.5000 | k2 = 9.4749 | k3 = 9.4751 | k4 = 9.4500
41 | y_prox = 97.5000 + (1.0/6) * (9.5000 + 2*9.4749 + 2*9.4751 + 9.4500) = 106.9750
42 | Iteracao 12:
43 | k1 = 9.4500 | k2 = 9.4249 | k3 = 9.4251 | k4 = 9.4000
44 | y_prox = 106.9750 + (1.0/6) * (9.4500 + 2*9.4249 + 2*9.4251 + 9.4000) = 116.4000
45 | Iteracao 13:
46 | k1 = 9.4000 | k2 = 9.3749 | k3 = 9.3751 | k4 = 9.3500
47 | y_prox = 116.4000 + (1.0/6) * (9.4000 + 2*9.3749 + 2*9.3751 + 9.3500) = 125.7750
48 | Iteracao 14:
49 | k1 = 9.3500 | k2 = 9.3249 | k3 = 9.3251 | k4 = 9.3000
50 | y_prox = 125.7750 + (1.0/6) * (9.3500 + 2*9.3249 + 2*9.3251 + 9.3000) = 135.1000
51 | Iteracao 15:
52 | k1 = 9.3000 | k2 = 9.2749 | k3 = 9.2751 | k4 = 9.2500
53 | y_prox = 135.1000 + (1.0/6) * (9.3000 + 2*9.2749 + 2*9.2751 + 9.2500) = 144.3750
54 | Iteracao 16:
55 | k1 = 9.2500 | k2 = 9.2249 | k3 = 9.2251 | k4 = 9.2000
56 | y_prox = 144.3750 + (1.0/6) * (9.2500 + 2*9.2249 + 2*9.2251 + 9.2000) = 153.6000
57 | Iteracao 17:
58 | k1 = 9.2000 | k2 = 9.1749 | k3 = 9.1751 | k4 = 9.1500
59 | y_prox = 153.6000 + (1.0/6) * (9.2000 + 2*9.1749 + 2*9.1751 + 9.1500) = 162.7750
60 | Iteracao 18:
61 | k1 = 9.1500 | k2 = 9.1249 | k3 = 9.1251 | k4 = 9.1000
62 | y_prox = 162.7750 + (1.0/6) * (9.1500 + 2*9.1249 + 2*9.1251 + 9.1000) = 171.9000
63 | Iteracao 19:
64 | k1 = 9.1000 | k2 = 9.0749 | k3 = 9.0751 | k4 = 9.0500
65 | y_prox = 171.9000 + (1.0/6) * (9.1000 + 2*9.0749 + 2*9.0751 + 9.0500) = 180.9750
66 | Iteracao 20:
67 | k1 = 9.0500 | k2 = 9.0249 | k3 = 9.0251 | k4 = 9.0000
68 | y_prox = 180.9750 + (1.0/6) * (9.0500 + 2*9.0249 + 2*9.0251 + 9.0000) = 190.0000
69 | Iteracao 21:
70 | k1 = 9.0000 | k2 = 8.9749 | k3 = 8.9751 | k4 = 8.9500
71 | y_prox = 190.0000 + (1.0/6) * (9.0000 + 2*8.9749 + 2*8.9751 + 8.9500) = 198.9750

```

```

72 Iteracao 22:
73   k1 = 8.9500 | k2 = 8.9249 | k3 = 8.9251 | k4 = 8.9000
74   y_prox = 198.9750 + (1.0/6) * (8.9500 + 2*8.9249 + 2*8.9251 + 8.9000) = 207.9000
75 Iteracao 23:
76   k1 = 8.9000 | k2 = 8.8749 | k3 = 8.8751 | k4 = 8.8500
77   y_prox = 207.9000 + (1.0/6) * (8.9000 + 2*8.8749 + 2*8.8751 + 8.8500) = 216.7750
78 Iteracao 24:
79   k1 = 8.8500 | k2 = 8.8249 | k3 = 8.8251 | k4 = 8.8000
80   y_prox = 216.7750 + (1.0/6) * (8.8500 + 2*8.8249 + 2*8.8251 + 8.8000) = 225.6000
81 Iteracao 25:
82   k1 = 8.8000 | k2 = 8.7749 | k3 = 8.7751 | k4 = 8.7500
83   y_prox = 225.6000 + (1.0/6) * (8.8000 + 2*8.7749 + 2*8.7751 + 8.7500) = 234.3750
84 Iteracao 26:
85   k1 = 8.7500 | k2 = 8.7249 | k3 = 8.7251 | k4 = 8.7000
86   y_prox = 234.3750 + (1.0/6) * (8.7500 + 2*8.7249 + 2*8.7251 + 8.7000) = 243.1000
87 Iteracao 27:
88   k1 = 8.7000 | k2 = 8.6749 | k3 = 8.6751 | k4 = 8.6500
89   y_prox = 243.1000 + (1.0/6) * (8.7000 + 2*8.6749 + 2*8.6751 + 8.6500) = 251.7750
90 Iteracao 28:
91   k1 = 8.6500 | k2 = 8.6249 | k3 = 8.6251 | k4 = 8.6000
92   y_prox = 251.7750 + (1.0/6) * (8.6500 + 2*8.6249 + 2*8.6251 + 8.6000) = 260.4000
93 Iteracao 29:
94   k1 = 8.6000 | k2 = 8.5749 | k3 = 8.5751 | k4 = 8.5500
95   y_prox = 260.4000 + (1.0/6) * (8.6000 + 2*8.5749 + 2*8.5751 + 8.5500) = 268.9750
96 Iteracao 30:
97   k1 = 8.5500 | k2 = 8.5249 | k3 = 8.5251 | k4 = 8.5000
98   y_prox = 268.9750 + (1.0/6) * (8.5500 + 2*8.5249 + 2*8.5251 + 8.5000) = 277.5000
99 Iteracao 31:
100   k1 = 8.5000 | k2 = 8.4749 | k3 = 8.4751 | k4 = 8.4500
101   y_prox = 277.5000 + (1.0/6) * (8.5000 + 2*8.4749 + 2*8.4751 + 8.4500) = 285.9750
102 Iteracao 32:
103   k1 = 8.4500 | k2 = 8.4249 | k3 = 8.4251 | k4 = 8.4000
104   y_prox = 285.9750 + (1.0/6) * (8.4500 + 2*8.4249 + 2*8.4251 + 8.4000) = 294.4000

```

```

105 Iteracao 33:
106     k1 = 8.4000 | k2 = 8.3749 | k3 = 8.3751 | k4 = 8.3500
107     y_prox = 294.4000 + (1.0/6) * (8.4000 + 2*8.3749 + 2*8.3751 + 8.3500) = 302.7750
108 Iteracao 34:
109     k1 = 8.3500 | k2 = 8.3249 | k3 = 8.3251 | k4 = 8.3000
110     y_prox = 302.7750 + (1.0/6) * (8.3500 + 2*8.3249 + 2*8.3251 + 8.3000) = 311.1000
111 Iteracao 35:
112     k1 = 8.3000 | k2 = 8.2749 | k3 = 8.2751 | k4 = 8.2500
113     y_prox = 311.1000 + (1.0/6) * (8.3000 + 2*8.2749 + 2*8.2751 + 8.2500) = 319.3750
114 Iteracao 36:
115     k1 = 8.2500 | k2 = 8.2249 | k3 = 8.2251 | k4 = 8.2000
116     y_prox = 319.3750 + (1.0/6) * (8.2500 + 2*8.2249 + 2*8.2251 + 8.2000) = 327.6000
117 Iteracao 37:
118     k1 = 8.2000 | k2 = 8.1749 | k3 = 8.1751 | k4 = 8.1500
119     y_prox = 327.6000 + (1.0/6) * (8.2000 + 2*8.1749 + 2*8.1751 + 8.1500) = 335.7750
120 Iteracao 38:
121     k1 = 8.1500 | k2 = 8.1249 | k3 = 8.1251 | k4 = 8.1000
122     y_prox = 335.7750 + (1.0/6) * (8.1500 + 2*8.1249 + 2*8.1251 + 8.1000) = 343.9000
123 Iteracao 39:
124     k1 = 8.1000 | k2 = 8.0749 | k3 = 8.0751 | k4 = 8.0500
125     y_prox = 343.9000 + (1.0/6) * (8.1000 + 2*8.0749 + 2*8.0751 + 8.0500) = 351.9750
126 Iteracao 40:
127     k1 = 8.0500 | k2 = 8.0249 | k3 = 8.0251 | k4 = 8.0000
128     y_prox = 351.9750 + (1.0/6) * (8.0500 + 2*8.0249 + 2*8.0251 + 8.0000) = 360.0000
129 Iteracao 41:
130     k1 = 8.0000 | k2 = 7.9749 | k3 = 7.9751 | k4 = 7.9500
131     y_prox = 360.0000 + (1.0/6) * (8.0000 + 2*7.9749 + 2*7.9751 + 7.9500) = 367.9750
132 Iteracao 42:
133     k1 = 7.9500 | k2 = 7.9249 | k3 = 7.9251 | k4 = 7.9000
134     y_prox = 367.9750 + (1.0/6) * (7.9500 + 2*7.9249 + 2*7.9251 + 7.9000) = 375.9000
135 Iteracao 43:
136     k1 = 7.9000 | k2 = 7.8749 | k3 = 7.8751 | k4 = 7.8500
137     y_prox = 375.9000 + (1.0/6) * (7.9000 + 2*7.8749 + 2*7.8751 + 7.8500) = 383.7750

138 Iteracao 44:
139     k1 = 7.8500 | k2 = 7.8249 | k3 = 7.8251 | k4 = 7.8000
140     y_prox = 383.7750 + (1.0/6) * (7.8500 + 2*7.8249 + 2*7.8251 + 7.8000) = 391.6000
141 Iteracao 45:
142     k1 = 7.8000 | k2 = 7.7749 | k3 = 7.7751 | k4 = 7.7500
143     y_prox = 391.6000 + (1.0/6) * (7.8000 + 2*7.7749 + 2*7.7751 + 7.7500) = 399.3750
144 Iteracao 46:
145     k1 = 7.7500 | k2 = 7.7249 | k3 = 7.7251 | k4 = 7.7000
146     y_prox = 399.3750 + (1.0/6) * (7.7500 + 2*7.7249 + 2*7.7251 + 7.7000) = 407.1000
147 Iteracao 47:
148     k1 = 7.7000 | k2 = 7.6749 | k3 = 7.6751 | k4 = 7.6500
149     y_prox = 407.1000 + (1.0/6) * (7.7000 + 2*7.6749 + 2*7.6751 + 7.6500) = 414.7750
150 Iteracao 48:
151     k1 = 7.6500 | k2 = 7.6249 | k3 = 7.6251 | k4 = 7.6000
152     y_prox = 414.7750 + (1.0/6) * (7.6500 + 2*7.6249 + 2*7.6251 + 7.6000) = 422.4000
153 Iteracao 49:
154     k1 = 7.6000 | k2 = 7.5749 | k3 = 7.5751 | k4 = 7.5500
155     y_prox = 422.4000 + (1.0/6) * (7.6000 + 2*7.5749 + 2*7.5751 + 7.5500) = 429.9750
156 Iteracao 50:
157     k1 = 7.5500 | k2 = 7.5249 | k3 = 7.5251 | k4 = 7.5000
158     y_prox = 429.9750 + (1.0/6) * (7.5500 + 2*7.5249 + 2*7.5251 + 7.5000) = 437.5000
159
160 =====
161 RESULTADO FINAL - PONTOS ENCONTRADOS:
162 Pontos: ( (0.0000, 0.0000), (1.0000, 9.9750), (2.0000, 19.9000), (3.0000, 29.7750), (4.0000, 39.6000), (5.0000, 49.3750), (6.0000, 59.1000), (7.
0000, 68.7750), (8.0000, 78.4000), (9.0000, 87.9750), (10.0000, 97.5000), (11.0000, 106.9750), (12.0000, 116.4000), (13.0000, 125.7750), (14.
0000, 135.1000), (15.0000, 144.3750), (16.0000, 153.6000), (17.0000, 162.7750), (18.0000, 171.9000), (19.0000, 180.9750), (20.0000, 190.0000),
(21.0000, 198.9750), (22.0000, 207.9000), (23.0000, 216.7750), (24.0000, 225.6000), (25.0000, 234.3750), (26.0000, 243.1000), (27.0000, 251.
7750), (28.0000, 260.4000), (29.0000, 268.9750), (30.0000, 277.5000), (31.0000, 285.9750), (32.0000, 294.4000), (33.0000, 302.7750), (34.0000,
311.1000), (35.0000, 319.3750), (36.0000, 327.6000), (37.0000, 335.7750), (38.0000, 343.9000), (39.0000, 351.9750), (40.0000, 360.0000), (41.
0000, 367.9750), (42.0000, 375.9000), (43.0000, 383.7750), (44.0000, 391.6000), (45.0000, 399.3750), (46.0000, 407.1000), (47.0000, 414.7750),
(48.0000, 422.4000), (49.0000, 429.9750), (50.0000, 437.5000) )
163 =====

```

Resposta 12.3:

A velocidade final calculada pelo programa foi exata: $v(50) = 437,50$.

Problema Teste 2 - Página 425 Questão 12.10

12.10 - O estudo sobre o crescimento da população é importante para uma variedade de problemas de engenharia. Um modelo simples, onde o crescimento está sujeito à hipótese de que a taxa de variação da população p é proporcional à população num instante t , isto é:

$$\frac{dp}{dt} = Gp ,$$

onde G é a taxa de variação (por ano). Suponha que no instante $t = 0$, uma ilha tem uma população de 10000 habitantes. Se $G = 0.075$ por ano, utilize o método de Euler para prever a população em $t = 20$ anos, usando comprimento de passo de 0.5 ano.

Entrada 1 - para Euler

```
1 0.075 * y
2 0
3 10000
4 0.5
5 40
```

Saída – Euler (Entrada 1)

```
1  ===== RELATORIO: METODO DE EULER =====
2
3  Equacao Diferencial: dy/dx = 0.075 * y
4  Condicoes Iniciais: x0 = 0.0, y0 = 10000.0
5  Passo (h) = 0.5 | Iteracoes (n) = 40
6
7  --- INICIANDO ITERACOES ---
8  Iteracao 0: x = 0.0000, y = 10000.0000
9  Iteracao 1: f(0.0000, 10000.0000) = 750.0000 -> y_prox = 10375.0000
10 Iteracao 2: f(0.5000, 10375.0000) = 778.1250 -> y_prox = 10764.0625
11 Iteracao 3: f(1.0000, 10764.0625) = 807.3047 -> y_prox = 11167.7148
12 Iteracao 4: f(1.5000, 11167.7148) = 837.5786 -> y_prox = 11586.5042
13 Iteracao 5: f(2.0000, 11586.5042) = 868.9878 -> y_prox = 12020.9981
14 Iteracao 6: f(2.5000, 12020.9981) = 901.5749 -> y_prox = 12471.7855
15 Iteracao 7: f(3.0000, 12471.7855) = 935.3839 -> y_prox = 12939.4774
16 Iteracao 8: f(3.5000, 12939.4774) = 970.4608 -> y_prox = 13424.7078
17 Iteracao 9: f(4.0000, 13424.7078) = 1006.8531 -> y_prox = 13928.1344
18 Iteracao 10: f(4.5000, 13928.1344) = 1044.6101 -> y_prox = 14450.4394
19 Iteracao 11: f(5.0000, 14450.4394) = 1083.7830 -> y_prox = 14992.3309
20 Iteracao 12: f(5.5000, 14992.3309) = 1124.4248 -> y_prox = 15554.5433
21 Iteracao 13: f(6.0000, 15554.5433) = 1166.5907 -> y_prox = 16137.8387
22 Iteracao 14: f(6.5000, 16137.8387) = 1210.3379 -> y_prox = 16743.0076
23 Iteracao 15: f(7.0000, 16743.0076) = 1255.7256 -> y_prox = 17370.8704
24 Iteracao 16: f(7.5000, 17370.8704) = 1302.8153 -> y_prox = 18022.2781
25 Iteracao 17: f(8.0000, 18022.2781) = 1351.6709 -> y_prox = 18698.1135
26 Iteracao 18: f(8.5000, 18698.1135) = 1402.3585 -> y_prox = 19399.2927
27 Iteracao 19: f(9.0000, 19399.2927) = 1454.9470 -> y_prox = 20126.7662
28 Iteracao 20: f(9.5000, 20126.7662) = 1509.5075 -> y_prox = 20881.5200
29 Iteracao 21: f(10.0000, 20881.5200) = 1566.1140 -> y_prox = 21664.5770
30 Iteracao 22: f(10.5000, 21664.5770) = 1624.8433 -> y_prox = 22476.9986
31 Iteracao 23: f(11.0000, 22476.9986) = 1685.7749 -> y_prox = 23319.8860
32 Iteracao 24: f(11.5000, 23319.8860) = 1748.9915 -> y_prox = 24194.3818
33 Iteracao 25: f(12.0000, 24194.3818) = 1814.5786 -> y_prox = 25101.6711
34 Iteracao 26: f(12.5000, 25101.6711) = 1882.6253 -> y_prox = 26042.9838
35 Iteracao 27: f(13.0000, 26042.9838) = 1953.2238 -> y_prox = 27019.5956
36 Iteracao 28: f(13.5000, 27019.5956) = 2026.4697 -> y_prox = 28032.8305
37 Iteracao 29: f(14.0000, 28032.8305) = 2102.4623 -> y_prox = 29084.0616
```

```

38 Iteracao 30: f(14.5000, 29084.0616) = 2181.3046 -> y_prox = 30174.7139
39 Iteracao 31: f(15.0000, 30174.7139) = 2263.1035 -> y_prox = 31306.2657
40 Iteracao 32: f(15.5000, 31306.2657) = 2347.9699 -> y_prox = 32480.2507
41 Iteracao 33: f(16.0000, 32480.2507) = 2436.0188 -> y_prox = 33698.2601
42 Iteracao 34: f(16.5000, 33698.2601) = 2527.3695 -> y_prox = 34961.9448
43 Iteracao 35: f(17.0000, 34961.9448) = 2622.1459 -> y_prox = 36273.0178
44 Iteracao 36: f(17.5000, 36273.0178) = 2720.4763 -> y_prox = 37633.2559
45 Iteracao 37: f(18.0000, 37633.2559) = 2822.4942 -> y_prox = 39044.5030
46 Iteracao 38: f(18.5000, 39044.5030) = 2928.3377 -> y_prox = 40508.6719
47 Iteracao 39: f(19.0000, 40508.6719) = 3038.1504 -> y_prox = 42027.7471
48 Iteracao 40: f(19.5000, 42027.7471) = 3152.0810 -> y_prox = 43603.7876
49
50 =====
51 RESULTADO FINAL - PONTOS ENCONTRADOS:
52 Pontos: ( (0.0000, 10000.0000), (0.5000, 10375.0000), (1.0000, 10764.0625), (1.5000, 11167.7148), (2.0000, 11586.5042), (2.5000, 12020.9981), (3.
0000, 12471.7855), (3.5000, 12939.4774), (4.0000, 13424.7078), (4.5000, 13928.1344), (5.0000, 14450.4394), (5.5000, 14992.3309), (6.0000, 15554.
5433), (6.5000, 16137.8387), (7.0000, 16743.0076), (7.5000, 17370.8704), (8.0000, 18022.2781), (8.5000, 18698.1135), (9.0000, 19399.2927), (9.
5000, 20126.7662), (10.0000, 20881.5200), (10.5000, 21664.5770), (11.0000, 22476.9986), (11.5000, 23319.8860), (12.0000, 24194.3818), (12.5000,
25101.6711), (13.0000, 26042.9838), (13.5000, 27019.5956), (14.0000, 28032.8305), (14.5000, 29084.0616), (15.0000, 30174.7139), (15.5000, 31306.
2657), (16.0000, 32480.2507), (16.5000, 33698.2601), (17.0000, 34961.9448), (17.5000, 36273.0178), (18.0000, 37633.2559), (18.5000, 39044.5030),
(19.0000, 40508.6719), (19.5000, 42027.7471), (20.0000, 43603.7876) )
53 =====

```

Resposta 12.10:

O valor da população final projetado pelo programa foi de aproximadamente $p(20) = 43.603,7876$ habitantes.

Problema Teste 3 - Página 426 Questão 12.16

12.16 - Engenheiros ambientais e biomédicos precisam frequentemente prever o resultado de uma relação predador-presa ou hospedeiro-parasita. Um modelo simples para esse tipo de relação é dado pelas equações de Lotka-Volterra:

$$\frac{dH}{dt} = g_1 H - d_1 P H ,$$

$$\frac{dP}{dt} = -d_2 P + g_2 P H .$$

onde H e P são, respectivamente, por exemplo, o número de hospedeiros e parasitas presentes. As constantes d e g representam as taxas de mortalidade e crescimento, respectivamente. O índice 1 refere-se ao hospedeiro e o índice 2 ao parasita. Observe que essas equações formam um sistema de equações acopladas. Sabendo que no instante $t_0 = 0$, os valores de P e H são, respectivamente, 5 e 20, e que $g_1 = 1$, $d_1 = 0.1$, $g_2 = 0.02$ e $d_2 = 0.5$, utilize um método numérico para calcular os valores de H e P de 0 até 2s, usando passo de 0.01s.

Entrada 1 – para Runge-Kutta de 4º ordem

```
1 | 1*y - 0.1*y*5
2 | 0
3 | 20
4 | 0.01
5 | 200
```

Entrada 2 – para Runge-Kutta de 4º ordem

```
1 | -0.5*y + 0.02*y*20
2 | 0
3 | 5
4 | 0.01
5 | 200
```

Saída – Runge-Kutta de 4º ordem (Entrada 1)

```
1  ===== RELATORIO: METODO DE RUNGE-KUTTA 4A ORDEM =====
2
3  Equacao Diferencial: dy/dx = 1*y - 0.1*y*5
4  Condições Iniciais: x0 = 0.0, y0 = 20.0
5  Passo (h) = 0.01 | Iteracoes (n) = 200
6
7  --- INICIANDO ITERACOES ---
8  Iteracao 0: x = 0.0000, y = 20.0000
9  Iteracao 1:
10     k1 = 10.0000 | k2 = 10.0250 | k3 = 10.0251 | k4 = 10.0501
11     y_prox = 20.0000 + (0.01/6) * (10.0000 + 2*10.0250 + 2*10.0251 + 10.0501) = 20.1003
12  Iteracao 2:
13     k1 = 10.0501 | k2 = 10.0753 | k3 = 10.0753 | k4 = 10.1005
14     y_prox = 20.1003 + (0.01/6) * (10.0501 + 2*10.0753 + 2*10.0753 + 10.1005) = 20.2010
15  Iteracao 3:
16     k1 = 10.1005 | k2 = 10.1258 | k3 = 10.1258 | k4 = 10.1511
17     y_prox = 20.2010 + (0.01/6) * (10.1005 + 2*10.1258 + 2*10.1258 + 10.1511) = 20.3023
18  Iteracao 4:
19     k1 = 10.1511 | k2 = 10.1765 | k3 = 10.1766 | k4 = 10.2020
20     y_prox = 20.3023 + (0.01/6) * (10.1511 + 2*10.1765 + 2*10.1766 + 10.2020) = 20.4040
21  Iteracao 5:
22     k1 = 10.2020 | k2 = 10.2275 | k3 = 10.2276 | k4 = 10.2532
23     y_prox = 20.4040 + (0.01/6) * (10.2020 + 2*10.2275 + 2*10.2276 + 10.2532) = 20.5063
24  Iteracao 6:
25     k1 = 10.2532 | k2 = 10.2788 | k3 = 10.2788 | k4 = 10.3045
26     y_prox = 20.5063 + (0.01/6) * (10.2532 + 2*10.2788 + 2*10.2788 + 10.3045) = 20.6091
27  Iteracao 7:
28     k1 = 10.3045 | k2 = 10.3303 | k3 = 10.3304 | k4 = 10.3562
29     y_prox = 20.6091 + (0.01/6) * (10.3045 + 2*10.3303 + 2*10.3304 + 10.3562) = 20.7124
30  Iteracao 8:
31     k1 = 10.3562 | k2 = 10.3821 | k3 = 10.3822 | k4 = 10.4081
32     y_prox = 20.7124 + (0.01/6) * (10.3562 + 2*10.3821 + 2*10.3822 + 10.4081) = 20.8162
33  Iteracao 9:
34     k1 = 10.4081 | k2 = 10.4341 | k3 = 10.4342 | k4 = 10.4603
35     y_prox = 20.8162 + (0.01/6) * (10.4081 + 2*10.4341 + 2*10.4342 + 10.4603) = 20.9206
36  Iteracao 10:
37     k1 = 10.4603 | k2 = 10.4864 | k3 = 10.4865 | k4 = 10.5127
```

```

606 Iteracao 200:
607 k1 = 27.0472 | k2 = 27.1149 | k3 = 27.1150 | k4 = 27.1828
608 y_prox = 54.0945 + (0.01/6) * (27.0472 + 2*27.1149 + 2*27.1150 + 27.1828) = 54.3656
609
610 =====
611 RESULTADO FINAL - PONTOS ENCONTRADOS:
612 Pontos: ( (0.0000, 20.0000), (0.0100, 20.1003), (0.0200, 20.2010), (0.0300, 20.3023), (0.0400, 20.4040), (0.0500, 20.5063), (0.0600, 20.6091),
(0.0700, 20.7124), (0.0800, 20.8162), (0.0900, 20.9206), (0.1000, 21.0254), (0.1100, 21.1308), (0.1200, 21.2367), (0.1300, 21.3432), (0.1400, 21.
4502), (0.1500, 21.5577), (0.1600, 21.6657), (0.1700, 21.7743), (0.1800, 21.8835), (0.1900, 21.9932), (0.2000, 22.1034), (0.2100, 22.2142), (0.
2200, 22.3256), (0.2300, 22.4375), (0.2400, 22.5499), (0.2500, 22.6630), (0.2600, 22.7766), (0.2700, 22.8907), (0.2800, 23.0055), (0.2900, 23.
1208), (0.3000, 23.2367), (0.3100, 23.3532), (0.3200, 23.4702), (0.3300, 23.5879), (0.3400, 23.7061), (0.3500, 23.8249), (0.3600, 23.9443), (0.
3700, 24.0644), (0.3800, 24.1850), (0.3900, 24.3062), (0.4000, 24.4281), (0.4100, 24.5505), (0.4200, 24.6736), (0.4300, 24.7972), (0.4400, 24.
9215), (0.4500, 25.0465), (0.4600, 25.1720), (0.4700, 25.2982), (0.4800, 25.4250), (0.4900, 25.5524), (0.5000, 25.6805), (0.5100, 25.8092), (0.
5200, 25.9386), (0.5300, 26.0686), (0.5400, 26.1993), (0.5500, 26.3306), (0.5600, 26.4626), (0.5700, 26.5952), (0.5800, 26.7285), (0.5900, 26.
8625), (0.6000, 26.9972), (0.6100, 27.1325), (0.6200, 27.2685), (0.6300, 27.4052), (0.6400, 27.5426), (0.6500, 27.6806), (0.6600, 27.8194), (0.
6700, 27.9588), (0.6800, 28.0990), (0.6900, 28.2398), (0.7000, 28.3814), (0.7100, 28.5236), (0.7200, 28.6666), (0.7300, 28.8103), (0.7400, 28.
9547), (0.7500, 29.0998), (0.7600, 29.2457), (0.7700, 29.3923), (0.7800, 29.5396), (0.7900, 29.6877), (0.8000, 29.8365), (0.8100, 29.9861), (0.
8200, 30.1364), (0.8300, 30.2874), (0.8400, 30.4392), (0.8500, 30.5918), (0.8600, 30.7452), (0.8700, 30.8993), (0.8800, 31.0541), (0.8900, 31.
2098), (0.9000, 31.3662), (0.9100, 31.5235), (0.9200, 31.6815), (0.9300, 31.8403), (0.9400, 31.9999), (0.9500, 32.1603), (0.9600, 32.3215), (0.
9700, 32.4835), (0.9800, 32.6463), (0.9900, 32.8100), (1.0000, 32.9744), (1.0100, 33.1397), (1.0200, 33.3058), (1.0300, 33.4728), (1.0400, 33.
6406), (1.0500, 33.8092), (1.0600, 33.9786), (1.0700, 34.1490), (1.0800, 34.3201), (1.0900, 34.4922), (1.1000, 34.6651), (1.1100, 34.8388), (1.
1200, 35.0135), (1.1300, 35.1890), (1.1400, 35.3653), (1.1500, 35.5426), (1.1600, 35.7208), (1.1700, 35.8998), (1.1800, 36.0798), (1.1900, 36.
2606), (1.2000, 36.4424), (1.2100, 36.6250), (1.2200, 36.8086), (1.2300, 36.9931), (1.2400, 37.1786), (1.2500, 37.3649), (1.2600, 37.5522), (1.
2700, 37.7404), (1.2800, 37.9296), (1.2900, 38.1197), (1.3000, 38.3108), (1.3100, 38.5029), (1.3200, 38.6958), (1.3300, 38.8898), (1.3400, 39.
0847), (1.3500, 39.2807), (1.3600, 39.4776), (1.3700, 39.6754), (1.3800, 39.8743), (1.3900, 40.0742), (1.4000, 40.2751), (1.4100, 40.4769), (1.
4200, 40.6798), (1.4300, 40.8837), (1.4400, 41.0887), (1.4500, 41.2946), (1.4600, 41.5016), (1.4700, 41.7096), (1.4800, 41.9187), (1.4900, 42.
1288), (1.5000, 42.3400), (1.5100, 42.5522), (1.5200, 42.7655), (1.5300, 42.9799), (1.5400, 43.1953), (1.5500, 43.4118), (1.5600, 43.6294), (1.
5700, 43.8481), (1.5800, 44.0679), (1.5900, 44.2888), (1.6000, 44.5108), (1.6100, 44.7339), (1.6200, 44.9582), (1.6300, 45.1835), (1.6400, 45.
4100), (1.6500, 45.6376), (1.6600, 45.8664), (1.6700, 46.0963), (1.6800, 46.3273), (1.6900, 46.5596), (1.7000, 46.7929), (1.7100, 47.0275), (1.
7200, 47.2632), (1.7300, 47.5001), (1.7400, 47.7382), (1.7500, 47.9775), (1.7600, 48.2180), (1.7700, 48.4597), (1.7800, 48.7026), (1.7900, 48.
9467), (1.8000, 49.1921), (1.8100, 49.4386), (1.8200, 49.6865), (1.8300, 49.9355), (1.8400, 50.1858), (1.8500, 50.4374), (1.8600, 50.6902), (1.
8700, 50.9443), (1.8800, 51.1996), (1.8900, 51.4563), (1.9000, 51.7142), (1.9100, 51.9734), (1.9200, 52.2339), (1.9300, 52.4958), (1.9400, 52.
7589), (1.9500, 53.0233), (1.9600, 53.2891), (1.9700, 53.5562), (1.9800, 53.8247), (1.9900, 54.0945), (2.0000, 54.3656) )

```

OBS.: as iterações número 10 até a número 199 não foram coladas, porque o arquivo de saída ficou extremamente longo por conta delas, e seria inviável mostrar todas aqui.

Saída – Runge-Kutta de 4º ordem (Entrada 2)

```
1  ===== RELATORIO: METODO DE RUNGE-KUTTA 4A ORDEM =====
2
3  Equacao Diferencial: dy/dx = -0.5*y + 0.02*y*20
4  Condições Iniciais: x0 = 0.0, y0 = 5.0
5  Passo (h) = 0.01 | Iteracoes (n) = 200
6
7  --- INICIANDO ITERACOES ---
8  Iteracao 0: x = 0.0000, y = 5.0000
9  ✓ Iteracao 1:
10     k1 = -0.5000 | k2 = -0.4997 | k3 = -0.4998 | k4 = -0.4995
11     y_prox = 5.0000 + (0.01/6) * (-0.5000 + 2*-0.4997 + 2*-0.4998 + -0.4995) = 4.9950
12  ✓ Iteracao 2:
13     k1 = -0.4995 | k2 = -0.4993 | k3 = -0.4993 | k4 = -0.4990
14     y_prox = 4.9950 + (0.01/6) * (-0.4995 + 2*-0.4993 + 2*-0.4993 + -0.4990) = 4.9900
15  ✓ Iteracao 3:
16     k1 = -0.4990 | k2 = -0.4988 | k3 = -0.4988 | k4 = -0.4985
17     y_prox = 4.9900 + (0.01/6) * (-0.4990 + 2*-0.4988 + 2*-0.4988 + -0.4985) = 4.9850
18  ✓ Iteracao 4:
19     k1 = -0.4985 | k2 = -0.4983 | k3 = -0.4983 | k4 = -0.4980
20     y_prox = 4.9850 + (0.01/6) * (-0.4985 + 2*-0.4983 + 2*-0.4983 + -0.4980) = 4.9800
21  ✓ Iteracao 5:
22     k1 = -0.4980 | k2 = -0.4978 | k3 = -0.4978 | k4 = -0.4975
23     y_prox = 4.9800 + (0.01/6) * (-0.4980 + 2*-0.4978 + 2*-0.4978 + -0.4975) = 4.9751
24  ✓ Iteracao 6:
25     k1 = -0.4975 | k2 = -0.4973 | k3 = -0.4973 | k4 = -0.4970
26     y_prox = 4.9751 + (0.01/6) * (-0.4975 + 2*-0.4973 + 2*-0.4973 + -0.4970) = 4.9701
27  ✓ Iteracao 7:
28     k1 = -0.4970 | k2 = -0.4968 | k3 = -0.4968 | k4 = -0.4965
29     y_prox = 4.9701 + (0.01/6) * (-0.4970 + 2*-0.4968 + 2*-0.4968 + -0.4965) = 4.9651
30  ✓ Iteracao 8:
31     k1 = -0.4965 | k2 = -0.4963 | k3 = -0.4963 | k4 = -0.4960
32     y_prox = 4.9651 + (0.01/6) * (-0.4965 + 2*-0.4963 + 2*-0.4963 + -0.4960) = 4.9602
33  ✓ Iteracao 9:
34     k1 = -0.4960 | k2 = -0.4958 | k3 = -0.4958 | k4 = -0.4955
35     y_prox = 4.9602 + (0.01/6) * (-0.4960 + 2*-0.4958 + 2*-0.4958 + -0.4955) = 4.9552
36  ✓ Iteracao 10:
37     k1 = -0.4955 | k2 = -0.4953 | k3 = -0.4953 | k4 = -0.4950
```

```

606 Iteracao 200:
607 k1 = -0.4098 | k2 = -0.4096 | k3 = -0.4096 | k4 = -0.4094
608 y_prox = 4.0977 + (0.01/6) * (-0.4098 + 2*-0.4096 + 2*-0.4096 + -0.4094) = 4.0937
609
610 =====
611 RESULTADO FINAL - PONTOS ENCONTRADOS:
612 Pontos: ( (0.0000, 5.0000), (0.0100, 4.9950), (0.0200, 4.9900), (0.0300, 4.9850), (0.0400, 4.9800), (0.0500, 4.9751), (0.0600, 4.9701), (0.0700,
4.9651), (0.0800, 4.9602), (0.0900, 4.9552), (0.1000, 4.9502), (0.1100, 4.9453), (0.1200, 4.9404), (0.1300, 4.9354), (0.1400, 4.9305), (0.1500,
4.9256), (0.1600, 4.9206), (0.1700, 4.9157), (0.1800, 4.9108), (0.1900, 4.9059), (0.2000, 4.9010), (0.2100, 4.8961), (0.2200, 4.8912), (0.2300,
4.8863), (0.2400, 4.8814), (0.2500, 4.8765), (0.2600, 4.8717), (0.2700, 4.8668), (0.2800, 4.8619), (0.2900, 4.8571), (0.3000, 4.8522), (0.3100,
4.8474), (0.3200, 4.8425), (0.3300, 4.8377), (0.3400, 4.8329), (0.3500, 4.8280), (0.3600, 4.8232), (0.3700, 4.8184), (0.3800, 4.8136), (0.3900,
4.8088), (0.4000, 4.8039), (0.4100, 4.7991), (0.4200, 4.7943), (0.4300, 4.7896), (0.4400, 4.7848), (0.4500, 4.7800), (0.4600, 4.7752), (0.4700,
4.7704), (0.4800, 4.7657), (0.4900, 4.7609), (0.5000, 4.7561), (0.5100, 4.7514), (0.5200, 4.7466), (0.5300, 4.7419), (0.5400, 4.7372), (0.5500,
4.7324), (0.5600, 4.7277), (0.5700, 4.7230), (0.5800, 4.7182), (0.5900, 4.7135), (0.6000, 4.7088), (0.6100, 4.7041), (0.6200, 4.6994), (0.6300,
4.6947), (0.6400, 4.6900), (0.6500, 4.6853), (0.6600, 4.6807), (0.6700, 4.6760), (0.6800, 4.6713), (0.6900, 4.6666), (0.7000, 4.6620), (0.7100,
4.6573), (0.7200, 4.6527), (0.7300, 4.6480), (0.7400, 4.6434), (0.7500, 4.6387), (0.7600, 4.6341), (0.7700, 4.6294), (0.7800, 4.6248), (0.7900,
4.6202), (0.8000, 4.6156), (0.8100, 4.6110), (0.8200, 4.6064), (0.8300, 4.6018), (0.8400, 4.5972), (0.8500, 4.5926), (0.8600, 4.5880), (0.8700,
4.5834), (0.8800, 4.5788), (0.8900, 4.5742), (0.9000, 4.5697), (0.9100, 4.5651), (0.9200, 4.5605), (0.9300, 4.5560), (0.9400, 4.5514), (0.9500,
4.5469), (0.9600, 4.5423), (0.9700, 4.5378), (0.9800, 4.5332), (0.9900, 4.5287), (1.0000, 4.5242), (1.0100, 4.5197), (1.0200, 4.5151), (1.0300,
4.5106), (1.0400, 4.5061), (1.0500, 4.5016), (1.0600, 4.4971), (1.0700, 4.4926), (1.0800, 4.4881), (1.0900, 4.4837), (1.1000, 4.4792), (1.1100,
4.4747), (1.1200, 4.4702), (1.1300, 4.4658), (1.1400, 4.4613), (1.1500, 4.4568), (1.1600, 4.4524), (1.1700, 4.4479), (1.1800, 4.4435), (1.1900,
4.4390), (1.2000, 4.4346), (1.2100, 4.4302), (1.2200, 4.4257), (1.2300, 4.4213), (1.2400, 4.4169), (1.2500, 4.4125), (1.2600, 4.4081), (1.2700,
4.4037), (1.2800, 4.3993), (1.2900, 4.3949), (1.3000, 4.3905), (1.3100, 4.3861), (1.3200, 4.3817), (1.3300, 4.3773), (1.3400, 4.3730), (1.3500,
4.3686), (1.3600, 4.3642), (1.3700, 4.3599), (1.3800, 4.3555), (1.3900, 4.3511), (1.4000, 4.3468), (1.4100, 4.3424), (1.4200, 4.3381), (1.4300,
4.3338), (1.4400, 4.3294), (1.4500, 4.3251), (1.4600, 4.3208), (1.4700, 4.3165), (1.4800, 4.3122), (1.4900, 4.3078), (1.5000, 4.3035), (1.5100,
4.2992), (1.5200, 4.2949), (1.5300, 4.2906), (1.5400, 4.2864), (1.5500, 4.2821), (1.5600, 4.2778), (1.5700, 4.2735), (1.5800, 4.2692), (1.5900,
4.2650), (1.6000, 4.2607), (1.6100, 4.2565), (1.6200, 4.2522), (1.6300, 4.2480), (1.6400, 4.2437), (1.6500, 4.2395), (1.6600, 4.2352), (1.6700,
4.2310), (1.6800, 4.2268), (1.6900, 4.2225), (1.7000, 4.2183), (1.7100, 4.2141), (1.7200, 4.2099), (1.7300, 4.2057), (1.7400, 4.2015), (1.7500,
4.1973), (1.7600, 4.1931), (1.7700, 4.1889), (1.7800, 4.1847), (1.7900, 4.1805), (1.8000, 4.1764), (1.8100, 4.1722), (1.8200, 4.1680), (1.8300,
4.1638), (1.8400, 4.1597), (1.8500, 4.1555), (1.8600, 4.1514), (1.8700, 4.1472), (1.8800, 4.1431), (1.8900, 4.1389), (1.9000, 4.1348), (1.9100,
4.1307), (1.9200, 4.1265), (1.9300, 4.1224), (1.9400, 4.1183), (1.9500, 4.1142), (1.9600, 4.1101), (1.9700, 4.1060), (1.9800, 4.1018), (1.9900,
4.0977), (2.0000, 4.0937) )
613 =====

```

OBS.: as iterações número 10 até a número 199 não foram coladas, porque o arquivo de saída ficou extremamente longo por conta delas, e seria inviável mostrar todas aqui.

Resposta 12.16:

Aplicando a aproximação de desacoplamento das variáveis através de suas condições iniciais, o algoritmo projetou as seguintes populações finais: para os hospedeiros, obteve-se aproximadamente $H(2) = 54,3656$; para os parasitas, obteve-se aproximadamente $P(2) = 4,0937$.

Método do Shooting (Tiro)

Estratégia de Implementação:

A implementação do Método do Shooting (Tiro) marcou a transição algorítmica para a resolução de Problemas de Valor de Contorno (PVC). A estratégia central consistiu em converter a equação diferencial de segunda ordem original em um sistema equivalente de duas equações de primeira ordem ($y' = z$ e $z' = f(x, y, z)$). Dessa forma, o problema de contorno é transitoriamente tratado como um Problema de Valor Inicial (PVI), exigindo que a derivada inicial desconhecida (z_0) seja descoberta por meio de tentativas iterativas até que a curva atinja o ponto final desejado (o "alvo").

O diferencial e o grau de sofisticação desta implementação residem na automação da busca por esse ângulo de disparo ideal. Em vez de depender de chutes arbitrários inseridos pelo usuário, o algoritmo calcula o primeiro disparo mirando diretamente no alvo, assumindo a inclinação de uma reta simples ($z_0 = (y_f - y_0) / (x_f - x_0)$). Após um segundo disparo com um leve incremento angular ($z_1 = z_0 + 0,1$), o programa aciona internamente o Método da Secante. Essa lógica iterativa realimenta os erros de distância (onde a curva caiu versus onde deveria cair) para ajustar automaticamente o ângulo de disparo a cada nova tentativa, garantindo uma convergência rápida e precisa.

Para traçar o percurso de cada um desses disparos de teste, bem como o trajeto final, a rotina foi integrada a uma função dedicada ao Método de Runge-Kutta de 4ª Ordem adaptada para sistemas (rk4_sistema). O interpretador dinâmico de strings foi expandido para processar equações que contêm x , y e a derivada z , avaliando simultaneamente o avanço de ambas as variáveis a cada passo de integração numérico. O laço do Método da Secante encerra-se apenas quando o erro final do percurso se torna menor ou igual à tolerância rigorosa estipulada.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): A leitura sequencial obedece à seguinte ordem obrigatória de sete linhas e sem rótulos textuais: expressão da EDO de 2ª ordem isolada em $y'' = f(x, y, z)$ (utilizando z no lugar de y'); valor numérico da coordenada inicial da variável independente (x_0); valor numérico da condição de

contorno inicial da variável dependente (y_0); valor numérico da coordenada final da variável independente (x_f); valor numérico da condição de contorno final esperada (y_f , atuando como o alvo); tamanho numérico do passo de incremento espacial (h); tolerância máxima de erro admissível para validar o acerto no alvo (tol).

Arquivo de Saída (saida.txt): O relatório deste método apresenta um memorial descritivo dividido em duas fases distintas, refletindo a natureza iterativa da abordagem. Na primeira fase, o arquivo registra o histórico do Método da Secante, imprimindo linha a linha a inclinação testada, a altura atingida e a margem de erro daquele disparo específico em relação ao alvo. Uma vez que a convergência é confirmada e o limite de tolerância é respeitado, a fase dois é ativada. O sistema imprime a evolução da curva passo a passo, exibindo os cálculos definitivos efetuados pelo núcleo do RK4 para o ângulo vencedor. Por fim, a totalidade das coordenadas do disparo validado é agrupada em uma malha contínua formatada em pares ordenados do tipo (x, y) , garantindo uma leitura padronizada do vetor de solução do PVC.

Dificuldades enfrentadas

A principal dificuldade técnica na implementação do Método do Shooting residuiu na orquestração simultânea de dois algoritmos numéricos distintos: a integração acoplada via Runge-Kutta de 4ª Ordem e a busca iterativa de raízes pelo Método da Secante. Como a EDO de segunda ordem original precisou ser decomposta em um sistema de variáveis de estado (y e z , onde $z = y'$), foi imperativo garantir que o avaliador dinâmico processasse as taxas de variação de ambas as equações estritamente em sincronia a cada passo fracionário do RK4. Além disso, a automação do ajuste do ângulo de disparo exigiu um rigoroso controle lógico nas iterações da Secante para prevenir falhas críticas de execução, sendo necessário implementar uma trava de segurança contra divisões por zero caso a diferença entre os erros de dois disparos consecutivos se anulasse. Assegurar a correta retroalimentação das variáveis de inclinação (z_0, z_1, z_2) a cada nova tentativa, garantindo a estabilidade da convergência geométrica até que o alvo y_f fosse atingido dentro da tolerância estabelecida, constituiu o maior desafio arquitetural desta rotina.

Método das Diferenças Finitas

Estratégia de Implementação:

A implementação do Método das Diferenças Finitas consolidou uma abordagem algébrica direta para a resolução de Problemas de Valor de Contorno (PVC) lineares. Em contraste com a natureza iterativa de tentativa e erro do Método do Tiro, esta estratégia fundamenta-se na discretização completa do domínio $[x_0, x_f]$ em uma malha de $n - 1$ pontos internos, substituindo as derivadas contínuas por aproximações de diferenças finitas centrais.

Para manter a premissa de não utilizar bibliotecas simbólicas externas, o algoritmo foi projetado para assumir que a EDO possui a forma linear $y'' = p(x)y' + q(x)y + r(x)$. A extração desses coeficientes a partir da string de texto fornecida pelo usuário foi arquitetada por meio de um artifício algébrico injetado no avaliador dinâmico: a cada ponto x_i , o sistema calcula r_i avaliando $f(x_i, 0, 0)$, descobre q_i processando $f(x_i, 1, 0) - r_i$ e obtém p_i por intermédio de $f(x_i, 0, 1) - r_i$.

Com os coeficientes dinamicamente mapeados, a rotina constrói um sistema linear de formato estritamente tridiagonal. As condições de contorno fixas nas extremidades (y_0 e y_f) são incorporadas diretamente ao vetor de termos independentes (d) na primeira e na última equação interna, condicionando perfeitamente a matriz. A resolução desse sistema dispensa o uso de pacotes pesados como o NumPy; em vez disso, foi desenvolvida uma função dedicada (`resolver_sistema_tridiagonal`) que aplica o veloz Algoritmo de Thomas (uma variação altamente otimizada da Eliminação de Gauss para matrizes tridiagonais), calculando o vetor de resposta através de uma fase de ida (fatoração) e uma fase de substituição regressiva.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (`entrada.txt`): A leitura sequencial obedece à seguinte ordem obrigatória de sete linhas e sem rótulos textuais: expressão da EDO de 2ª ordem isolada em $y'' = f(x, y, z)$ (utilizando z no lugar de y'); valor numérico da coordenada inicial da variável independente (x_0); valor numérico da condição de contorno inicial da variável dependente (y_0); valor numérico da coordenada final da variável independente (x_f); valor numérico da condição de contorno final esperada

(y_f , atuando como o alvo); tamanho numérico do passo de incremento espacial (h); tolerância máxima de erro admissível para validar o acerto no alvo (tol) - que, embora ociosa neste método analítico direto, é lida para preservar a compatibilidade de interface com o Método do Tiro.

Arquivo de Saída (saida.txt): O relatório de execução abandona o formato iterativo do método anterior para atuar como um memorial de álgebra linear dividido em três fases lógicas. Inicialmente, o documento expõe a extração exata dos coeficientes temporais $p(x)$, $q(x)$ e $r(x)$ para cada ponto interno avaliado. Na segunda fase, o arquivo imprime a montagem do sistema tridiagonal, detalhando os multiplicadores das diagonais e os termos independentes ajustados pelas condições de contorno em formato de equação por extenso. Após a resolução silenciosa pelo Algoritmo de Thomas, a terceira fase aglutina as fronteiras fixas originais aos novos pontos internos descobertos, apresentando a resposta final em uma malha contínua formatada em pares ordenados do tipo (x, y) .

Dificuldades enfrentadas

A principal dificuldade técnica na implementação do Método das Diferenças Finitas residiu na abstração algébrica necessária para generalizar a extração dos coeficientes $p(x)$, $q(x)$ e $r(x)$ sem o auxílio de bibliotecas de computação simbólica. Foi imperativo desenvolver um artifício lógico robusto integrado ao avaliador dinâmico, forçando o programa a deduzir as proporções da EDO em tempo de execução: a função $f(x, y, z)$ teve de ser testada repetidamente a cada ponto da malha com combinações de variáveis zeradas ou unitárias (por exemplo, obtendo r_i ao avaliar $f(x_i, 0, 0)$ e isolando q_i e p_i por subtração direta). Adicionalmente, a montagem do sistema linear tridiagonal exigiu precisão matemática rigorosa no gerenciamento dos índices dos vetores e no tratamento das bordas do domínio. Assegurar que as condições de contorno fixas (y_0 e y_f) fossem alocadas e transferidas corretamente para o vetor de termos independentes nas equações das extremidades, evitando falhas estruturais ou erros de acesso (index out of range) durante as varreduras do Algoritmo de Thomas, constituiu o maior desafio para garantir o condicionamento da matriz e a confiabilidade da resposta final.

Problema Teste 4 - Questão fornecida no slide do professor

Problema de transferência de calor em uma haste, proposto para ser resolvido com 10 pontos internos.

Entrada 1 - para Shooting

```
1 0.01 * (y - 20)
2 0
3 40
4 10
5 200
6 0.9090909090909091
7 0.0001
```

Saída – Shooting (Entrada 1)

```
1  ===== RELATORIO: METODO DO TIRO (SHOOTING) =====
2
3  EDO de 2a Ordem:  $y'' = 0.01 * (y - 20)$ 
4  Contorno Inicial:  $y(0.0) = 40.0$ 
5  Contorno Final (Alvo):  $y(10.0) = 200.0$ 
6  Passo (h) = 0.9090909090909091 | Tolerancia = 0.0001
7
8  --- FASE 1: CALIBRANDO A MIRA (Metodo da Secante) ---
9  Tiro 1: Inclinação inicial ( $z_0$ ) = 16.0000 | Atingiu  $y(10.0)$  = 238.8937 | Erro = 38.8937
10 Tiro 2: Inclinação inicial ( $z_1$ ) = 16.1000 | Atingiu  $y(10.0)$  = 240.0689 | Erro = 40.0689
11 Tiro 3: Inclinação inicial ajustada ( $z$ ) = 12.6905 | Atingiu  $y$  = 200.0000 | Erro = -0.0000
12
13 => ALVO ATINGIDO! Inclinação ideal encontrada:  $z = 12.6905$ 
14
15 --- GRAVANDO TRAJETO DO TIRO CERTEIRO (Passo a Passo RK4) ---
16 Iteracao 0:  $x = 0.0000$ ,  $y = 40.0000$ ,  $z(y') = 12.6905$ 
17 Iteracao 1:  $x = 0.9091 \rightarrow y_{prox} = 51.6354$ 
18 Iteracao 2:  $x = 1.8182 \rightarrow y_{prox} = 63.5324$ 
19 Iteracao 3:  $x = 2.7273 \rightarrow y_{prox} = 75.7894$ 
20 Iteracao 4:  $x = 3.6364 \rightarrow y_{prox} = 88.5078$ 
21 Iteracao 5:  $x = 4.5455 \rightarrow y_{prox} = 101.7928$ 
22 Iteracao 6:  $x = 5.4545 \rightarrow y_{prox} = 115.7542$ 
23 Iteracao 7:  $x = 6.3636 \rightarrow y_{prox} = 130.5076$ 
24 Iteracao 8:  $x = 7.2727 \rightarrow y_{prox} = 146.1748$ 
25 Iteracao 9:  $x = 8.1818 \rightarrow y_{prox} = 162.8855$ 
26 Iteracao 10:  $x = 9.0909 \rightarrow y_{prox} = 180.7779$ 
27 Iteracao 11:  $x = 10.0000 \rightarrow y_{prox} = 200.0000$ 
28
29 =====
30 RESULTADO FINAL - PONTOS ENCONTRADOS:
31 Pontos: ( (0.0000, 40.0000), (0.9091, 51.6354), (1.8182, 63.5324), (2.7273, 75.7894),
32 (3.6364, 88.5078), (4.5455, 101.7928), (5.4545, 115.7542), (6.3636, 130.5076),
33 (7.2727, 146.1748), (8.1818, 162.8855), (9.0909, 180.7779), (10.0000, 200.0000) )
34 =====
```

Problema Teste 5 - Questão fornecida no slide do professor

Problema de transferência de calor em uma haste, proposto para ser resolvido com 10 pontos internos, agora com o outro método.

Entrada 1 - para Diferenças Finitas

```
1 0.01 * (y - 20)
2 0
3 40
4 10
5 200
6 0.9090909090909091
7 0.0001
```

Saída – Diferenças Finitas (Entrada 1)

```
1  ===== RELATORIO: METODO DIFERENCAS FINITAS =====
2
3  EDO de 2a Ordem:  $y'' = 0.01 * (y - 20)$ 
4  Contorno Inicial:  $y(0.0) = 40.0$ 
5  Contorno Final (Alvo):  $y(10.0) = 200.0$ 
6  Passo (h) = 0.9090909090909091 | Tolerancia = 0.0001
7
8  --- FASE 1: EXTRAINDO COEFICIENTES p(x), q(x), r(x) ---
9  Assumindo EDO linear da forma:  $y'' = p(x)*y' + q(x)*y + r(x)$ 
10
11 --- FASE 2: MONTANDO O SISTEMA TRIDIAGONAL ---
12 Ponto interno 1 (x = 0.9091): p=0.0000, q=0.0100, r=-0.2000
13   Eq:  $(-1.0000)*y_0 + (2.0083)*y_1 + (-1.0000)*y_2 = 40.1653$ 
14 Ponto interno 2 (x = 1.8182): p=0.0000, q=0.0100, r=-0.2000
15   Eq:  $(-1.0000)*y_1 + (2.0083)*y_2 + (-1.0000)*y_3 = 0.1653$ 
16 Ponto interno 3 (x = 2.7273): p=0.0000, q=0.0100, r=-0.2000
17   Eq:  $(-1.0000)*y_2 + (2.0083)*y_3 + (-1.0000)*y_4 = 0.1653$ 
18 Ponto interno 4 (x = 3.6364): p=0.0000, q=0.0100, r=-0.2000
19   Eq:  $(-1.0000)*y_3 + (2.0083)*y_4 + (-1.0000)*y_5 = 0.1653$ 
20 Ponto interno 5 (x = 4.5455): p=0.0000, q=0.0100, r=-0.2000
21   Eq:  $(-1.0000)*y_4 + (2.0083)*y_5 + (-1.0000)*y_6 = 0.1653$ 
22 Ponto interno 6 (x = 5.4545): p=0.0000, q=0.0100, r=-0.2000
23   Eq:  $(-1.0000)*y_5 + (2.0083)*y_6 + (-1.0000)*y_7 = 0.1653$ 
24 Ponto interno 7 (x = 6.3636): p=0.0000, q=0.0100, r=-0.2000
25   Eq:  $(-1.0000)*y_6 + (2.0083)*y_7 + (-1.0000)*y_8 = 0.1653$ 
26 Ponto interno 8 (x = 7.2727): p=0.0000, q=0.0100, r=-0.2000
27   Eq:  $(-1.0000)*y_7 + (2.0083)*y_8 + (-1.0000)*y_9 = 0.1653$ 
28 Ponto interno 9 (x = 8.1818): p=0.0000, q=0.0100, r=-0.2000
29   Eq:  $(-1.0000)*y_8 + (2.0083)*y_9 + (-1.0000)*y_{10} = 0.1653$ 
30 Ponto interno 10 (x = 9.0909): p=0.0000, q=0.0100, r=-0.2000
31   Eq:  $(-1.0000)*y_9 + (2.0083)*y_{10} + (-1.0000)*y_{11} = 200.1653$ 
32
33 --- FASE 3: RESOLVENDO O SISTEMA (Algoritmo de Thomas) ---
34
35 =====
36 RESULTADO FINAL - PONTOS ENCONTRADOS:
37 Pontos: ( (0.0000, 40.0000), (0.9091, 51.6372), (1.8182, 63.5358),
38 (2.7273, 75.7943), (3.6364, 88.5138), (4.5455, 101.7996), (5.4545, 115.7614),
39 (6.3636, 130.5146), (7.2727, 146.1812), (8.1818, 162.8906), (9.0909, 180.7809),
40 (10.0000, 200.0000) )
41 =====
```

Problema Teste 6 - Modelo tipo SEIAHR simples para simulação de quadro epidêmico hipotético

Analisando a evolução do vírus no conjunto da população fornecida ao longo de 50 dias, usando o modelo SEIAHR simples com os parâmetros propostos, cheguei na seguinte tabela:

Dia	Suscetíveis	Expostos	Infectados	Hospitalizados	Óbitos
0	400000.0	0.0	1.0	0.00	0.00
5	399999.1	0.5	0.7	0.00	0.00
10	399997.9	0.7	0.6	0.00	0.00
15	399996.3	1.0	0.6	0.00	0.00
20	399994.0	1.3	0.8	0.00	0.00
25	399990.8	1.9	1.0	0.01	0.00
30	399986.4	2.6	1.3	0.01	0.00
35	399980.3	3.6	1.8	0.01	0.01
40	399971.7	5.1	2.5	0.01	0.01
45	399959.8	7.1	3.5	0.02	0.01
50	399943.2	9.9	4.9	0.03	0.01

Como podemos ver, comecei com apenas 1 infectado no dia 0, porém, como a variável de quarentena estava em 60%, ou seja, essa porcentagem de pessoas não estavam expostas e estavam com protocolos seguros, e 75% das pessoas que contraem são assintomáticas, com isso não entrando na coluna de infectados, o vírus mal se espalhou. Nesse contexto, sabendo que ele obedece uma função exponencial, vemos que o mesmo ficou no início da curva, quase não fez estrago na sociedade, por conta desses fatores de atraso. E ao término dos 50 dias, só foram computados 4,9 infectados sintomáticos ativos, 9,9 pacientes expostos (vírus ainda se multiplicando no organismo, não sentem nada e ainda não transmitem), e com o número de perdas (óbitos) em 0,01, praticamente 0, com demanda hospitalar muito baixa, 0,03 leitos ocupado.

Considerações Finais

A realização deste trabalho proporcionou uma imersão profunda na aplicação prática de métodos numéricos para a resolução de Equações Diferenciais Ordinárias (EDOs). Ao longo do desenvolvimento computacional, foi possível transpor a teoria matemática para algoritmos funcionais, lidando inicialmente com Problemas de Valor Inicial (PVI). Através das resoluções de dinâmicas populacionais, interações predador-presa e simulações físicas de resistência, ficou evidente como as ferramentas numéricas são estruturas fundamentais para prever o comportamento de sistemas naturais que fogem das soluções analíticas triviais.

Na avaliação comparativa dos algoritmos, observou-se uma clara distinção de eficiência estrutural. Enquanto o Método de Euler se manteve como uma ferramenta didática e direta para aproximações de primeira ordem, as rígidas exigências de precisão dos problemas reais tornaram indispensável a adoção de abordagens superiores. O Método de Runge-Kutta de 4ª Ordem consolidou-se como o motor matemático mais robusto deste relatório, entregando uma exatidão notável para o rastreamento das curvas e mitigando o acúmulo de erros de truncamento, justificando amplamente o seu leve acréscimo de custo computacional iterativo.

A transição para os Problemas de Valor de Contorno (PVC) ampliou a complexidade lógica da modelagem, o que ficou perfeitamente ilustrado pela simulação de transferência de calor em uma haste. A aplicação do Método do Shooting demonstrou a engenhosidade de converter contornos em problemas de valor inicial, acoplando a integração do RK4 a algoritmos de busca de raízes, como o da Secante, para calibrar iterativamente a derivada inicial. Em contrapartida, o Método das Diferenças Finitas revelou o poder da álgebra linear direta, transformando o domínio contínuo em uma malha discreta e solucionando o gradiente térmico de forma extremamente estável via matrizes tridiagonais, evidenciando como estratégias distintas podem romper a mesma barreira física.

O ápice analítico do estudo, contudo, materializou-se na simulação do quadro epidêmico hipotético da Covid-19, utilizando o modelo compartimental SEIAHR. Este desafio sublinhou a importância crítica dos métodos numéricos no planejamento de saúde pública, exigindo a resolução de um sistema denso de sete dimensões acopladas. A adaptação vetorial do algoritmo numérico permitiu visualizar

claramente a resposta do sistema às variáveis externas, como o drástico achatamento da curva exponencial provocado pela adoção da quarentena. Ficou claro como o rigor matemático — exemplificado pela necessária normalização da força de infecção frente à população total — é a linha tênue que separa uma projeção realista de um colapso por explosão numérica.

Em suma, as implementações desenvolvidas ratificam que não existe um algoritmo universalmente superior, mas sim a ferramenta metodológica mais adequada para cada contexto e limitação computacional. O equilíbrio entre precisão geométrica, viabilidade de codificação algorítmica e custo de processamento deve nortear as grandes decisões da análise de sistemas. Este trabalho não apenas consolidou as fundamentações teóricas da Análise Numérica, como também comprovou, de forma incontestável, a capacidade da computação de traduzir a complexidade do mundo real em matrizes, equações e, finalmente, respostas aplicáveis.